

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 2

Student Name: __Naïma Tahirou Maïyaki

Student ID: _____ 30252027 _____

GitHub Repository Link: _____

Date Submitted: _____ 7/3/2026 _____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	yes
Estimated percentage of the work influenced by AI	20%
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
ChatGPT	I used ChatGPT to understand the lab instructions, explain machine-learning concepts, and get help organizing my code.	“Explain this section,” “Show the code,” “Help me understand the output”	Data cleaning, feature engineering, train/validation/test split, regression, classification, K-Means clustering, graphs.	I ran all code myself in Jupyter Notebook and checked the outputs. I corrected errors when they appeared.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	no	n/a
Prompt history	no	n/a
Screenshot of AI interaction	yes	n/a
Exported AI conversation	no	n/a
AI-generated code snippet	no	n/a
Revised version showing student input	no	n/a

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: _____Naïma Tahirou Maïyaki

Date: _____7/3/2026_____

Appendixes:

explain it, do not provide the code

Section 1 — Supervised Learning: Regression

Predicting taxi tip_amount (NYC Yellow Taxi)

Each row is a completed taxi trip. Your goal is to predict the tip a passenger leaves (tip_amount, a continuous value) from the trip's characteristics. The available columns are:

VendorID, passenger_count, trip_distance, RatecodeID,
store_and_fwd_flag, PULocationID, DOLocationID, payment_type,
fare_amount, mta_tax, tolls_amount, improvement_surcharge,
tip_amount

Show more ▾



This is the first practical task: **understand the taxi data before building a model.**

Each row represents one completed taxi trip. The column `tip_amount` is what the passenger tipped. Your future model will use the other trip details to estimate that amount.

For example:



- `passenger_count` tells you how many people were in the taxi.

```
explain
TODO: Handle missing / invalid rows
# e.g. drop or impute NaNs; remove rows where trip_distance <= 0 or
fare_amount <= 0;
# decide how to treat negative tip_amount values.

# TODO: Feature engineering — create at least ONE new, justified
feature, for example:
# - tip_rate proxy, fare-per-mile = fare_amount / trip_distance
# - total surcharges = mta_tax + tolls_amount +
improvement_surcharge
Show more ▾
```



This part is about preparing the data so that a machine-learning model can use it correctly.

1. Handle missing or invalid rows

First, you check for missing values. Your output showed there are no missing values, so you do not need to drop rows or fill empty values.

Next, you remove or investigate invalid records:



+ Ask anything



Next, you remove or investigate invalid records:

- `trip_distance <= 0`: a completed taxi trip should normally have a distance greater than zero. Your data has a minimum trip distance of 0, so those rows need attention.
- `fare_amount <= 0`: a taxi trip with zero or negative fare is likely invalid for predicting tips, so those rows should be removed.
- Negative `tip_amount`: if they existed, they would be invalid because a passenger cannot leave a negative tip. Your output shows the minimum tip is 0.01, so there are no negative tips to remove.

2. Feature engineering

Feature engineering means creating a new useful column from existing columns.

You must create at least one feature and explain why it could help predict tips. A good choice is **fare per mile**:

- It is calculated from `fare_amount` and `trip_distance`.
- It shows how expensive the trip was relative to its distance.
- Two trips can have the same fare but different distances, so this feature can help the model identify trips affected by traffic, route type, airport rates, or other pricing conditions.

Another possible feature is **total surcharges**, combining `mta_tax`, `tolls_amount`, and `improvement_surcharge`. It represents extra charges added to the base fare.

Do **not** create a feature using `tip_amount`, such as tip rate. `tip_amount` is the answer the model is supposed to predict. If you use it inside a feature, the model indirectly sees the correct answer before making a prediction. This is called **target leakage**, and it gives misleadingly high performance.



3. Identify categorical and numeric columns

3. Identify categorical and numeric columns

Some values look numerical but are actually labels or categories.

Categorical columns include:

- `VendorID` : identifies the taxi company/vendor.
- `RatecodeID` : identifies the rate type.
- `store_and_fwd_flag` : represents a yes/no type of flag.
- `PULocationID` : pickup location ID.
- `DOLocationID` : drop-off location ID.
- `payment_type` : payment method category.

These should not be treated as normal numbers. For example, location ID 200 is not “twice as much” as location ID 100. They need encoding, usually by creating separate 0/1 columns for each category. This is called one-hot encoding.

Numeric columns include:

- `passenger_count`
- `trip_distance`
- `fare_amount`
- `mta_tax`
- `tolls_amount`
- `improvement_surcharge`
- your engineered feature, such as `fare_per_mile` ↓

4. Scale numerical features

4. Scale numerical features

Numeric columns use different units and ranges. For example, `trip_distance` can be close to 0 or as high as 189.91, while `mta_tax` is usually around 0.50. Scaling puts numeric features onto a comparable scale so that models are not overly influenced by columns with larger values.

You will use `StandardScaler` for this. However, you must wait until after the train/validation/test split in Part 1.3.

The scaler must learn the mean and standard deviation from the **training set only**, then apply the same transformation to validation and test data. If it uses validation or test data while learning, that is data leakage.

📄 👍 💬 ↗️ ↺️ ...

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
Cell In[4], line 1  
----> 1 from sklearn.model_selection import train_test_split  
      2 from sklearn.preprocessing import StandardScaler  
      3  
      4 # TODO: Separate features (X) and target (y = tip_amount)  
  
ModuleNotFoundError: No module named 'sklearn'
```

The notebook is still using the old Python kernel. The package installed in `ai_env`, but your notebook kernel is connected to a different Python environment.

Do this carefully:

1. Click **Select Kernel** at the top right of the notebook.
2. Choose **Python Environments**.
3. Select the interpreter whose path includes:



```
linear_model = LinearRegression()  
  
forest_model.fit(X_train_scaled, y_train)  
  
# TODO: Predict on train, validation, and test sets.  
linear_train_pred = linear_model.predict(X_train_scaled)  
linear_val_pred = linear_model.predict(X_val_scaled)  
linear_test_pred = linear_model.predict(X_test_scaled)  
  
forest_train_pred = forest_model.predict(X_train_scaled)  
  
Show more ▾
```



You are missing one important line:

Python



Run

```
linear_model.fit(X_train_scaled, y_train)
```

You created `linear_model`, but you did not train it. If you try to predict now, it will give an error.



Replace your full cell with this complete version:

You are missing one important line:

Python

```
linear_model.fit(X_train_scaled, y_train)
```



Run

You created `linear_model`, but you did not train it. If you try to predict now, it will give an error.

Replace your full cell with this complete version:

Python

```
# TODO 1: Train the regression models
linear_model = LinearRegression()
linear_model.fit(X_train_scaled, y_train)
```

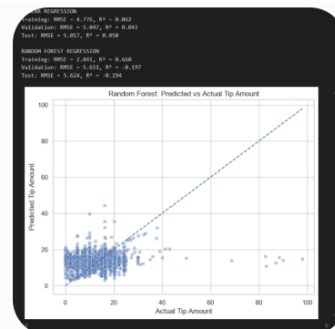
```
forest_model = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
forest_model.fit(X_train_scaled, y_train)
```



Run



```
# TODO 2: Predict on train, validation, and test sets
linear_model.predict(X_train_scaled)
```



help me understand the output

Your models are not performing well, but the output is useful because it clearly shows the difference between a simple model and an overfitted model.

RMSE shows the typical prediction error in dollars. Lower is better. R^2 shows how much of the variation in tip amounts the model explains. A value close to 1 is good; 0 means the model is not better than simply predicting the average tip; a negative value means it is worse than predicting the average.

For Linear Regression:



- Training RMSE = 4.776, validation RMSE = 5.047, and test RMSE = 5.017.

Ask anything



break it down

Part 2.2 — Preprocessing & feature engineering

Models need numeric input. Encode categoricals, scale numerics, and optionally add a feature.

TODO: Encode categorical columns.

- Binary yes/no columns (family_history_with_overweight, FAVC, SMOKE, SCC) -> 0/1

- Ordinal-ish columns (CAEC, CALC) and nominal (Gender, MTRANS) -> choose an encoding

and justify it (LabelEncoder / OrdinalEncoder / pd.get_dummies /

Show more ▾

This section prepares the obesity data for a classification model. There are **four TODOs**, but the last one happens later in Part 2.3.

TODO 1 — Encode the categorical input columns

Machine-learning models cannot use text values such as Female, yes, Sometimes, or Public Transportation directly. You must convert them into numbers.

- family_history_with_overweight
- FAVC
- SMOKE
- SCC

For these columns, use 0 and 1:

- no becomes 0
- yes becomes 1

For CAEC and CALC, the values show frequency, such as no, Sometimes, Frequently, and Always. These have a natural order from less frequent to more frequent. You can use ordinal encoding, where the categories become ordered numbers.

For Gender and MTRANS, use one-hot encoding. Gender and transport type are categories, but there is no meaningful numeric order between them. For example, Walking is not greater than Bike, and Male is not greater than Female. One-hot encoding creates separate 0/1 columns for each category.

TODO 2 — Create BMI, optional

BMI is calculated using:

$$BMI = \frac{Weight}{Height^2}$$



However, BMI may make the task very easy because obesity levels are often defined using BMI ranges. It is not exactly target leakage because BMI is calculated only from input features (`Weight` and `Height`), not from `NObesidad` . But it may make the prediction less realistic if the goal is to predict obesity level without using measurements that directly determine it.

For this lab, create BMI because the instructions encourage feature engineering, but explain this point in your reasoning.

TODO 3 — Encode the target

The target `NObesidad` contains seven text labels, such as `Normal_Weight` and `Obesity_Type_I` . The classifier needs numeric class labels instead.

Use `LabelEncoder` to change the seven labels into integers from 0 to 6. These numbers are only labels; they do not mean that one obesity category is mathematically bigger than another.

TODO 4 — Scale numeric features

Do not scale the numeric columns yet.

You will scale columns such as `Age` , `Height` , `Weight` , `FCVC` , `NCP` , `CH2O` , `FAF` , `TUE` , and BMI later in Part 2.3, after splitting the data into training, validation, and test sets.

You will fit the scaler on the training set only, then use it to transform validation and test data.

[show the complete code of part 2.2](#)

```
Python

# =====
# Part 2.2 - Preprocessing & Feature Engineering
# =====

from sklearn.preprocessing import LabelEncoder

# =====
# TODO 1: Encode categorical input columns
# =====

# Make a copy so the original obesity DataFrame stays unchanged
obesity_clean = obesity.copy()

# Binary yes/no columns: no = 0, yes = 1
binary_columns = [
    "family_history_with_overweight",
    "FAVC",
    "SMOKE",
    "SCC"
]

for column in binary_columns:
    obesity_clean[column] = obesity_clean[column].map({
        "no": 0,
        "yes": 1
    })
```

```
Traceback (most recent call last):
  File "<ipython_input>:1", line 1, in <module>
    obesity_clean = obesity.copy()
  File "/usr/local/lib/python3.7/dist-packages/numpy/core/_add_newdocs.py", line 599, in __call__
    return self._call(*args, **kwargs)
  File "/usr/local/lib/python3.7/dist-packages/numpy/core/_add_newdocs.py", line 605, in _call
    return getattr(self._obj, attrname)(*args, **kwargs)
ValueError: setting an array element with a sequence
```

The error happened because `obesity` is no longer a DataFrame. Earlier, you accidentally ran:

```
Python
obesity = target_encoder.fit_transform(...)
```

That changed `obesity` into a NumPy array of encoded numbers. So now this line fails:

```
Python
obesity_clean = obesity.copy()
```

because `obesity_clean` becomes an array, and arrays cannot use column names like `"family_history_with_overweight"`.

You need to reload the original obesity dataset first.

Run this new cell before Part 2.2:

```
Python
obesity = pd.read_csv(OBESITY_URL)
print(obesity.shape)
```

what are the importations supposed to be done for each cell?

You should import each tool in the first cell where you use it. You do not need to import the same thing again later.

- Setup cell

```
Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

- Part 1.3: Taxi split and scaling

```
Python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

- Part 1.4: Taxi regression

```
Python
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

- Part 2.2: Obesity preprocessing

+ Ask anything

